

Operativita

Backup e restore

Copie di sicurezza e ripristino del database quando qualcosa va storto.

Il backup è la rete di sicurezza del database

Un backup è una copia di sicurezza. Il restore è il processo con cui torni a quella copia quando ne hai bisogno.

Una regola semplice ma importantissima: **un backup vale davvero solo se sai ripristinarlo.**

Pensa a una foto importante salvata solo sul telefono. Dire "la copierò prima o poi" non ti protegge. Una copia vera, conservata bene e verificata, sì.

Perché non basta dire "abbiamo i backup"

Molti problemi nascono non dall'assenza di backup, ma dal fatto che nessuno ha mai provato seriamente il restore.

Quando arriva il momento del bisogno, scoprire che il recupero non funziona è troppo tardi.

Cosa tenere in mente

Un buon approccio include:

- fare backup con regolarità
- sapere dove sono conservati
- proteggere anche i backup
- testare periodicamente il ripristino

Un controllo semplice

Per ragionare bene sui backup, chiediti:

- quanto spesso faccio una copia?
- quanti dati posso permettermi di perdere?
- quanto tempo posso restare fermo?

- chi sa davvero fare il restore?

:::caution I backup contengono dati veri. Vanno protetti come il database principale. :::

Best practice SQL

Abitudini sane quando scrivi schema, query e manutenzione del database.

Le buone pratiche sono piccole scelte che fanno risparmiare molto dopo

In SQL la qualità raramente nasce da un colpo di genio. Nasce più spesso da abitudini sane ripetute con costanza.

Pensa alla manutenzione di una bicicletta. Gonfiare le gomme, controllare i freni e pulire la catena sembrano gesti piccoli. Insieme evitano grossi problemi.

- usa nomi chiari
- evita `SELECT *` quando non serve
- aggiungi vincoli
- testa `UPDATE` e `DELETE` con una `SELECT` prima
- fai backup regolari

Alcune abitudini da adottare subito

Scrivi query leggibili:

```
SELECT nome, email
FROM clienti
WHERE attivo = TRUE;
```

Questa versione dice chiaramente quali colonne vuoi, da dove arrivano e quale filtro usi.

Il valore vero di queste abitudini

Ogni singola pratica può sembrare piccola. Insieme, però, rendono il database più leggibile, più stabile e meno fragile.

Le best practice non servono a sembrare esperti. Servono a farti trovare meno problemi domani.

Un buon modo per usarle

Non serve inseguirle come una checklist perfetta in ogni momento. Serve usarle come bussola mentre progetti, scrivi e mantieni query e schema.

:::caution La pratica più importante con `UPDATE` e `DELETE` è controllare prima il filtro. Sono comandi utili, ma possono fare molti danni se usati senza attenzione. :::

Anti-pattern comuni

Errori ricorrenti da evitare quando lavori con SQL e con la struttura del database.

Un anti-pattern è una scorciatoia che poi presenta il conto

Un anti-pattern è una soluzione che all'inizio sembra comoda, ma nel tempo genera confusione, bug o lentezza.

È come mettere tutto in un cassetto "varie". Oggi fai prima. Tra un mese non trovi più niente.

Esempi comuni:

- tabelle senza chiave primaria
- duplicazioni inutili
- colonne che mischiano troppi significati
- query lunghissime senza alias chiari

Un esempio facile da riconoscere

Una colonna chiamata `dati_extra` che contiene pezzi diversi di informazione può sembrare comoda.

Il problema arriva quando vuoi cercare, filtrare o controllare quei dati. Il database non sa più bene cosa rappresenta quella colonna.

Perché vale la pena riconoscerli

Se impari a vedere questi schemi presto, eviti di costruire problemi che diventeranno costosi solo più avanti.

Riconoscere un anti-pattern è già metà della correzione.

Una postura utile

Quando una soluzione sembra fin troppo comoda, vale la pena chiedersi se stai semplificando davvero oppure stai solo spostando il problema più avanti nel tempo.

:::tip Se una scelta rende difficile spiegare il database a voce, spesso merita una seconda occhiata. :::

Import/export dati

Scambiare dati con CSV, JSON e altri formati in entrata o in uscita.

Un database non vive da solo

Un database comunica spesso con altri sistemi. Per questo è normale importare dati dentro il database o esportarli verso file e strumenti esterni.

Per questo è comune importare o esportare dati in formati come:

- CSV
- JSON
- fogli di calcolo

Pensa al database come a un magazzino. A volte riceve scatole da fuori. A volte prepara pacchi da spedire ad altri.

Quando succede nella pratica

Per esempio:

- carichi un CSV ricevuto da un fornitore
- esporti dati per un report
- trasferisci informazioni tra sistemi diversi

La cosa da non sottovalutare

Importare ed esportare non significa solo "spostare file". Vuol dire anche controllare formati, codifiche, separatori e qualità dei dati.

Un esempio di attenzione pratica

Un file CSV può sembrare semplice, ma può nascondere dettagli importanti:

- quale carattere separa le colonne?
- le date sono nel formato giusto?
- i testi hanno accenti e caratteri speciali?
- ci sono righe vuote o valori mancanti?

:::tip Prima importa un piccolo campione. Se tutto torna, passa al file completo. :::

Sicurezza del database

Buone pratiche per proteggere dati, accessi e operazioni sensibili.

La sicurezza non è un argomento separato dal database: ne fa parte

La sicurezza del database non è un accessorio finale. Fa parte del lavoro quotidiano, proprio come scrivere query corrette o progettare tabelle sensate.

Pensa a un archivio con documenti personali. Non basta ordinarlo bene. Devi anche decidere chi può aprirlo, copiarlo o modificarlo.

Buone abitudini:

- dare solo i permessi necessari
- evitare account condivisi
- proteggere i dati sensibili
- usare credenziali robuste

Una mentalità utile

Non pensare solo a "come entro". Pensa anche a:

- chi può vedere cosa
- chi può modificare cosa
- come proteggere i backup
- come registrare accessi ed errori importanti

Un database custodisce informazioni. Proteggerle è parte del suo mestiere.

Un primo principio semplice

Il principio del minimo privilegio dice: dai a ogni utente solo i permessi che gli servono davvero.

Se un'app deve solo leggere dati pubblici, non dovrebbe poter cancellare tabelle.

:::caution Anche i backup vanno protetti. Spesso contengono gli stessi dati sensibili del database principale. :::

Versioning e migrazioni database

Tenere traccia dei cambiamenti dello schema nel tempo senza perdere il controllo.

Lo schema cambia, e quei cambiamenti vanno trattati con disciplina

Le migrazioni sono cambiamenti controllati dello schema. Aggiungere una colonna, creare una tabella, modificare un vincolo: tutto questo dovrebbe lasciare una traccia chiara.

Versionarle aiuta il team a sapere sempre quale struttura deve avere il database.

Pensa alle versioni di un regolamento condominiale. Se ognuno ha una copia diversa, nascono discussioni. Se ogni modifica ha data e ordine, tutti capiscono cosa vale.

Perché è così importante

Senza versioning, ogni ambiente rischia di diventare diverso dagli altri. Uno sviluppatore ha una colonna in più, un altro no, e iniziano confusione ed errori.

Le migrazioni portano ordine dove altrimenti regnerebbe memoria personale e improvvisazione.

Un esempio semplice

Una migrazione può dire:

```
ALTER TABLE clienti  
ADD telefono VARCHAR(30);
```

Questa modifica aggiunge la colonna `telefono`. Salvandola in una migrazione, puoi applicarla in modo ordinato anche su altri ambienti.

Una buona abitudine da adottare presto

Anche in progetti piccoli, tenere traccia dei cambiamenti dello schema è un investimento utile.

Ti evita di affidarti al ricordo e rende il database parte integrante del lavoro di squadra.

:::caution Cambiare o cancellare colonne con dati reali richiede prudenza. Prima pensa a backup, compatibilità e piano di ritorno. :::

ETL

Extract, Transform, Load spiegato come un flusso pratico di lavoro sui dati.

ETL è il viaggio dei dati da una sorgente a una destinazione

ETL significa:

1. **Extract:** prendi i dati
2. **Transform:** li sistemi
3. **Load:** li carichi nel database

Pensa a un pacco che arriva in magazzino. Prima lo scarichi, poi controlli e sistemi il contenuto, infine lo metti nello scaffale giusto.

Un esempio concreto

Immagina di ricevere ogni giorno un file CSV da un altro sistema.

Prima lo leggi, poi sistemi i valori sporchi o incompleti, infine carichi i dati nel database. **Questo è ETL.**

Cosa succede nella fase di trasformazione

Durante **Transform** potresti:

- correggere date scritte in formati diversi
- eliminare righe duplicate
- convertire testi in maiuscolo o minuscolo
- controllare valori mancanti

Perché è importante

È un concetto molto comune quando i dati arrivano da sorgenti diverse e non sono subito pronti per entrare nel tuo sistema.

Il valore vero dell'ETL

Non è solo spostare dati da A a B. È fare in modo che arrivino puliti, coerenti e nel formato giusto.

Questo fa una grande differenza quando i sistemi coinvolti diventano molti.

:::note ETL non è un singolo comando SQL. È un modo di organizzare un processo. :::

EXPLAIN e query plan

Capire come il database decide di eseguire una query e dove perde tempo.

Quando una query è lenta, non si indovina: si guarda il piano

EXPLAIN serve a vedere il piano di esecuzione di una query. In altre parole, ti mostra come il database pensa di arrivare al risultato.

È come chiedere a un navigatore: "che strada pensi di fare?" Prima di cambiare percorso, vuoi vedere il tragitto scelto.

```
EXPLAIN
SELECT *
FROM ordini
WHERE totale > 100;
```

Questa query non serve a mostrare gli ordini. Serve a mostrare come il database intende cercarli.

Perché è così utile

Guardare il query plan aiuta a capire se:

- il database sta leggendo troppe righe
- manca un indice utile
- una join è costosa
- la query può essere scritta meglio

Non ti dà una soluzione automatica, ma ti dà la mappa del problema.

Cosa guardare all'inizio

Quando sei alle prime armi, cerca soprattutto segnali semplici:

- il database legge tutta la tabella?
- usa un indice?
- stima tantissime righe?

:::tip Non serve capire ogni riga del piano subito. Inizia a usarlo come una lente per fare domande migliori. :::

JSON e dati semi-strutturati

Lavorare con campi JSON dentro un database relazionale senza perdere la bussola.

Non tutti i dati sono rigidi come una tabella classica

Alcuni dati hanno una parte stabile e una parte più flessibile. I campi JSON servono quando vuoi conservare informazioni con struttura meno rigida dentro un database relazionale.

SQL resta relazionale, ma molti database moderni sanno gestire anche dati semi-strutturati.

Pensa a una scheda prodotto. Nome, prezzo e codice sono campi stabili. Le caratteristiche tecniche, invece, possono cambiare molto da prodotto a prodotto.

Quando può essere utile

Per esempio:

- impostazioni personalizzate di un utente
- metadati variabili
- configurazioni che cambiano da caso a caso

Il punto da ricordare

JSON è utile, ma non dovrebbe diventare una scusa per buttare tutto in un campo indistinto. **Se un dato è davvero strutturato e importante, spesso merita ancora una colonna o una tabella dedicata.**

Un esempio piccolo

Un campo `preferenze` potrebbe contenere dati come:

```
{
  "tema": "scuro",
  "lingua": "it"
}
```

Queste preferenze possono cambiare nel tempo senza obbligarti subito a modificare molte colonne.

La scelta giusta è quasi sempre mista

Molti progetti usano una base relazionale solida e aggiungono JSON solo dove la flessibilità serve davvero.

Questo equilibrio di solito è più sano di un approccio tutto o niente.

Gestione di grandi dataset

Idee pratiche quando i dati diventano molto numerosi e le abitudini piccole non bastano più.

Quando i dati crescono, cambiano anche le domande giuste

Con pochi dati, quasi tutto sembra veloce. Quando i dati crescono davvero, iniziano a contare scelte che prima sembravano secondarie.

È come cucinare per quattro persone o per quattrocento. La ricetta può essere simile, ma organizzazione, tempi e strumenti cambiano molto.

Diventano importanti:

- indici ben scelti
- query efficienti
- partizionamento
- monitoraggio
- archiviazione dei dati vecchi

La lezione principale

Gestire grandi dataset non significa solo avere hardware migliore. Significa soprattutto progettare meglio query, struttura e processi.

La scala grande punisce le scorciatoie piccole.

Un esempio pratico

Una query che legge tutti gli ordini può andare bene con 500 righe.

Con 50 milioni di righe, la stessa query può diventare un problema. A quel punto servono filtri, indici, archiviazione e magari partizionamento.

Cosa cambia nella mentalità

Con molti dati diventa ancora più importante pensare in anticipo a manutenzione, osservabilità e crescita futura.

Non basta che qualcosa funzioni oggi. Deve continuare a funzionare bene anche quando il volume aumenta molto.

:::tip Quando progetti una tabella, chiediti anche come verrà letta tra un anno, non solo come viene scritta oggi. :::

Logging e monitoring

Tenere sotto controllo salute, errori e carico del database nel tempo.

Se non osservi il database, lavori quasi al buio

Logging e monitoring servono a capire cosa sta succedendo davvero. Quando il sistema rallenta, fallisce o si comporta in modo strano, hai bisogno di tracce affidabili.

- **logging:** registra eventi ed errori
- **monitoring:** osserva salute, tempi e carico

Perché sono diversi

Il logging racconta episodi specifici. Il monitoring ti mostra l'andamento generale.

Uno è più vicino al diario degli eventi. L'altro è più simile al cruscotto di un'auto.

Cosa potresti osservare

In un database può essere utile controllare:

- query lente
- errori frequenti
- spazio su disco
- numero di connessioni
- tempi medi di risposta

Perché sono indispensabili

Senza visibilità, i problemi arrivano prima di essere capiti. **Con buoni segnali, puoi intervenire prima che un piccolo rallentamento diventi un problema serio.**

La loro utilità cresce con il sistema

Più il progetto cresce, più diventa importante poter ricostruire cosa è successo e quando.

Questo vale sia per la diagnosi tecnica sia per la serenità operativa del team.

:::tip Non aspettare il primo problema grave per attivare logging e monitoring. Prepararli prima è molto più tranquillo. :::

Introduzione a ORM

Object Relational Mapping spiegato senza gergo inutile.

Un ORM è un ponte tra il tuo codice e il database

ORM significa **Object Relational Mapping**. In pratica è uno strumento che ti permette di lavorare con il database usando oggetti o classi del tuo linguaggio, senza scrivere sempre SQL a mano.

Pensa a un cameriere che prende il tuo ordine e lo porta in cucina. Tu non parli direttamente con ogni cuoco, ma la cucina esiste comunque e deve lavorare bene.

Un ORM fa qualcosa di simile: il tuo codice parla con l'ORM, e l'ORM prepara le query per il database.

Un esempio mentale

Invece di scrivere a mano:

```
SELECT nome, email
FROM clienti
WHERE id = 1;
```

con un ORM potresti usare un metodo del tuo linguaggio, per esempio "trova il cliente con id 1".

Il risultato sembra più comodo, ma sotto il database riceve comunque una query.

Perché non sostituisce SQL

Un ORM può rendere comode molte operazioni ripetitive. Ma sotto c'è comunque un database relazionale, con query, join, filtri e performance da capire.

Sapere SQL resta fondamentale, anche quando usi un ORM. In molti casi è proprio ciò che ti permette di capire quando l'ORM ti sta aiutando e quando invece ti sta nascondendo un problema.

Il modo migliore di vederlo

Un ORM è un assistente, non un sostituto della comprensione.

Se sai già cosa significa fare una join, filtrare con `WHERE` o raggruppare con `GROUP BY`, allora usi l'ORM con molta più consapevolezza.

:::tip Quando qualcosa è lento, prova sempre a guardare la query SQL generata dall'ORM. Spesso il problema diventa molto più chiaro. :::

Analisi delle performance

Come ragionare sulle query lente e sui colli di bottiglia senza andare a tentoni.

Migliorare le performance significa osservare, non indovinare

Quando qualcosa è lento, il primo riflesso giusto è misurare. Non basta dire "mi sembra lenta". Bisogna capire dove si perde tempo.

È come sentire un rumore strano in auto. Prima di cambiare pezzi a caso, apri il cofano, ascolti, controlli e cerchi la causa.

Controlla soprattutto:

- query lente
- tabelle grandi
- indici mancanti
- join costose
- uso inutile di `SELECT *`

La domanda giusta

Non chiederti solo: "come faccio a renderla veloce?"

Chiediti prima: **"cosa sta rallentando davvero?"**

È questo il passo che separa una correzione utile da un tentativo casuale.

Un piccolo percorso pratico

Quando una query è lenta, puoi procedere così:

1. guarda quanto tempo impiega
2. controlla quante righe legge
3. osserva il piano con `EXPLAIN`
4. verifica se manca un indice utile
5. riscrivi la query solo dopo aver capito il problema

:::caution Una query veloce con 100 righe può diventare lenta con 10 milioni di righe. I test devono assomigliare il più possibile alla realtà. :::

Un'abitudine molto utile

Guarda il problema nel contesto: volume dei dati, frequenza della query, presenza di indici, piano di esecuzione e pattern di utilizzo.

Le performance non si migliorano bene guardando un solo pezzo isolato.

Sharding

Il concetto generale di dividere i dati tra server diversi.

Qui non dividi solo una tabella: dividi il carico su più macchine

Lo sharding distribuisce i dati su più server. Invece di tenere tutto in una sola macchina, dividi il carico in più parti.

Pensa a una catena di negozi. Se un solo negozio non riesce a servire tutti i clienti, apri più sedi. Ogni sede gestisce una parte del lavoro.

Perché si fa

Quando un sistema cresce moltissimo, un solo server può non bastare più. In questi casi si può distribuire il carico e i dati su più nodi.

Per esempio, potresti mettere alcuni clienti su un server e altri clienti su un altro server, seguendo una regola precisa.

Il prezzo del vantaggio

Aiuta a scalare sistemi molto grandi, ma aumenta parecchio la complessità tecnica e operativa.

Per questo è un concetto da conoscere, ma non una soluzione da applicare troppo presto.

Con lo sharding diventano più difficili domande come:

- dove si trova questo dato?
- come faccio una query che attraversa più server?
- come faccio backup e manutenzione?
- cosa succede se uno shard ha molto più traffico degli altri?

La lezione pratica

Prima di pensare allo sharding, di solito conviene migliorare schema, indici, query e infrastruttura normale.

Molti progetti non ne avranno mai bisogno, ed è perfettamente normale.

Differenze tra dialetti SQL

MySQL, PostgreSQL, SQLite e SQL Server a confronto in modo semplice.

SQL è una lingua comune con accenti diversi

SQL è uno standard, ma ogni database ha il suo accento. Le idee di base sono molto simili, però alcuni dettagli cambiano.

Pensa all'italiano parlato in regioni diverse. La lingua è la stessa, ma certe parole, espressioni e abitudini cambiano.

Controlla sempre:

- tipi di dati
- auto-increment
- funzioni disponibili
- sintassi di `LIMIT` o equivalenti
- supporto a feature avanzate

Perché conta per chi studia

All'inizio può sembrare fastidioso. In realtà è normale. **Prima impari la logica comune, poi impari le differenze del database che stai usando.**

Questo approccio evita di confondere il concetto con la variante.

Un esempio semplice

Per limitare i risultati, molti database usano:

```
SELECT nome  
FROM clienti  
LIMIT 10;
```

SQL Server usa spesso una forma diversa, come `TOP`.

Non cambia l'idea: vuoi solo vedere pochi risultati.

La buona notizia

Una volta capiti bene i concetti centrali, passare da un dialetto all'altro è molto più facile di quanto sembri.

Di solito cambi alcuni dettagli, non l'intero modo di ragionare.

SQL vs NoSQL

Quando ha senso usare SQL e quando può avere senso un approccio NoSQL.

Non sono due squadre rivali: sono strumenti diversi

SQL e NoSQL sono strumenti diversi, non squadre rivali. La scelta dipende da come sono fatti i dati e da cosa devi farci.

Pensa a due tipi di contenitori. Una cassettera è perfetta per oggetti ordinati per categoria. Uno scatolone flessibile è più comodo quando gli oggetti cambiano forma spesso.

SQL è spesso ottimo quando:

- i dati hanno struttura chiara
- servono relazioni forti
- le transazioni contano molto

NoSQL può essere utile quando la struttura è più flessibile o il modello di accesso è molto particolare.

Un esempio concreto

Un gestionale ordini ha spesso bisogno di tabelle collegate, vincoli e transazioni affidabili. Qui SQL è una scelta molto naturale.

Un sistema che salva eventi molto vari, con campi che cambiano spesso, potrebbe invece valutare un database NoSQL.

Il punto importante per chi inizia

Capire bene SQL ti aiuta anche a capire meglio NoSQL. Se conosci bene tabelle, relazioni e vincoli, capisci più facilmente cosa stai scegliendo di tenere e cosa stai scegliendo di sacrificare in altri modelli.

La domanda migliore da farsi

Invece di chiederti "qual è il migliore in assoluto?", chiediti:

"quale modello si adatta meglio ai dati e ai problemi che ho davanti?"

:::note In molti progetti reali convivono più strumenti. Non devi trasformare questa scelta in una gara. :::

Partizionamento delle tabelle

Dividere una tabella grande in parti più piccole per gestirla meglio.

Quando una tabella diventa troppo grande, dividerla può aiutare

Il partizionamento divide una tabella molto grande in sezioni più piccole. Per esempio puoi separare i dati per mese, per anno o per area geografica.

Questo può aiutare su dataset grandi e query mirate.

È come dividere le bollette in raccoglitori per anno. Se cerchi una bolletta del 2025, non apri tutti i raccoglitori della casa.

Perché funziona

Se la query cerca solo un pezzo del totale, il database può evitare di attraversare tutto.

Non è una bacchetta magica, ma in certi scenari è uno strumento molto utile.

Un esempio mentale

Una tabella `ordini` può essere divisa per anno:

- `ordini_2024`
- `ordini_2025`
- `ordini_2026`

Dal punto di vista dell'utente puoi continuare a pensare a una tabella unica, ma il database sa che i dati sono organizzati in parti.

Quando non basta da solo

Il partizionamento non sostituisce query scritte bene, indici sensati e una buona progettazione.

Aiuta molto, ma funziona davvero bene quando si combina con altre scelte intelligenti.

:::caution Il partizionamento aggiunge complessità. Ha senso quando il volume dei dati è davvero grande o quando hai un motivo operativo chiaro. :::

Testing di query SQL

Verificare che le query facciano davvero ciò che ti aspetti, non solo ciò che speri.

Anche le query meritano test, non solo il codice dell'applicazione

Una query può essere sintatticamente corretta e comunque fare la cosa sbagliata. Per questo anche le query vanno testate.

È come una calcolatrice: può darti un numero, ma devi comunque sapere se hai scritto l'operazione giusta.

È utile avere:

- dati di prova
- risultati attesi
- controlli automatici dove possibile

Cosa stai verificando davvero

Quando testi una query, stai controllando:

- che selezioni le righe giuste
- che escluda quelle sbagliate
- che faccia i calcoli corretti
- che continui a funzionare quando cambiano i dati di contesto

Testare significa ridurre sorprese, non perdere tempo.

Un esempio piccolo

Se una query deve trovare solo clienti attivi, prepara dati di prova con:

- un cliente attivo
- un cliente non attivo
- un cliente con dati mancanti

Poi controlla che nel risultato compaia solo chi deve comparire.

:::tip I casi strani sono preziosi. Spesso sono proprio loro a scoprire errori nascosti. :::

Gestione utenti e permessi

Controllare chi può leggere, scrivere o amministrare il database.

Un database serio non è una stanza aperta a tutti

Gestire utenti e permessi significa decidere chi può fare cosa. Non tutte le persone o applicazioni che usano un database devono avere gli stessi poteri.

Pensa alle chiavi di un edificio. Non dai la chiave della cassaforte a chi deve solo entrare nella sala riunioni.

Di solito si gestiscono:

- utenti
- ruoli
- permessi di lettura
- permessi di scrittura
- permessi amministrativi

Perché conta così tanto

Se dai troppi permessi a tutti, aumenti il rischio di:

- errori accidentali
- modifiche indesiderate
- accessi impropri

La regola più sana è dare a ciascuno solo ciò che serve davvero.

Un esempio semplice

Un'applicazione che mostra un catalogo prodotti potrebbe avere solo il permesso di leggere.

Un pannello amministrativo, invece, potrebbe avere anche il permesso di modificare prezzi e descrizioni.

:::note Dare meno permessi non significa fidarsi meno delle persone. Significa ridurre il danno possibile se qualcosa va storto. :::